

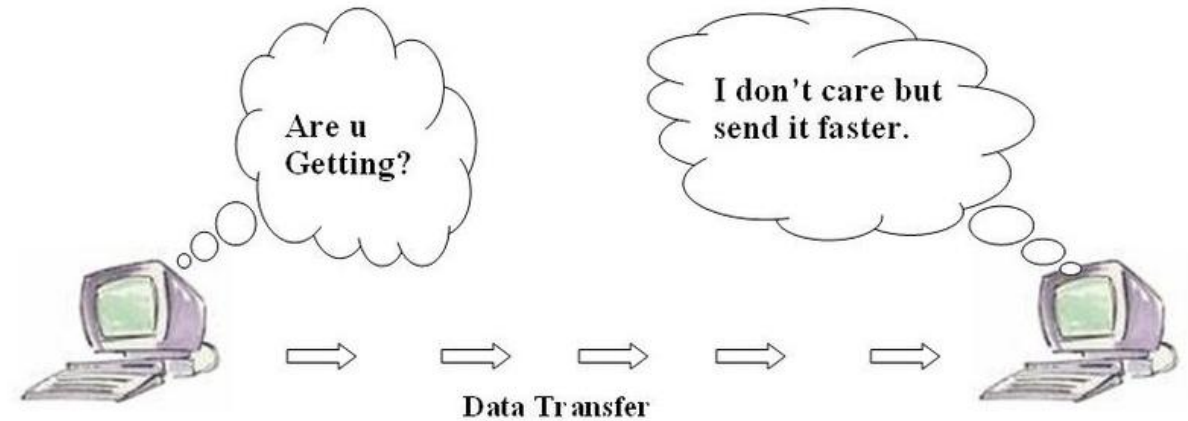
Computer Networks II

User Datagram Protocol

Amitangshu Pal
Computer Science and Engineering
IIT Kanpur

UDP: User Datagram Protocol

- “No frills,” “bare bones” Internet transport protocol
- “Best effort” service, UDP segments may be:
 - Lost, duplicated
 - Delivered out-of-order to app



Src: https://commons.wikimedia.org/wiki/File:Fig2_UDPwork.jpg

- **Connectionless:**
 - No handshaking between UDP sender, receiver
 - Each UDP segment handled independently of others

UDP: User Datagram Protocol

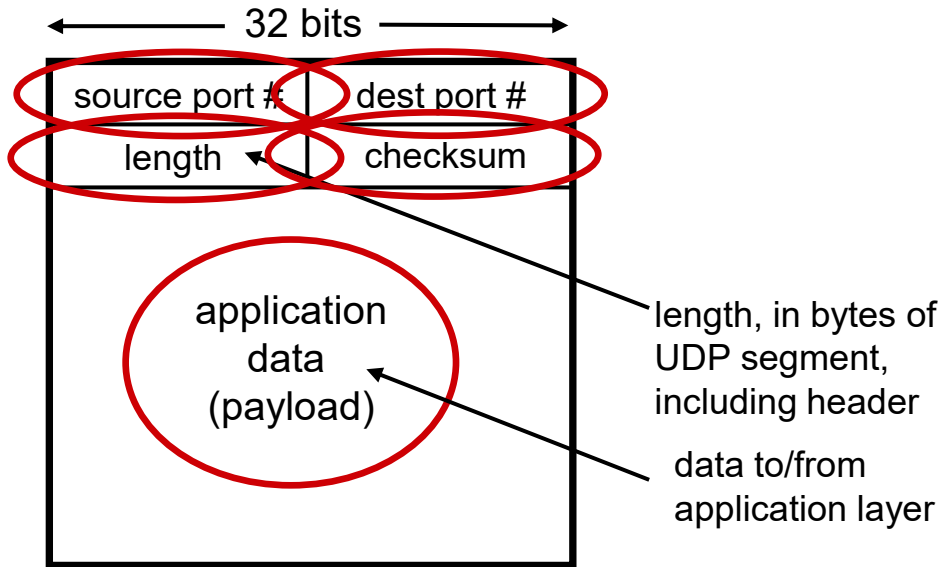
Why is there a UDP?

- No connection establishment (which can add RTT delay)
- Simple: no connection state at sender, receiver
- Small header size
- No congestion control
 - UDP can blast away as fast as desired!

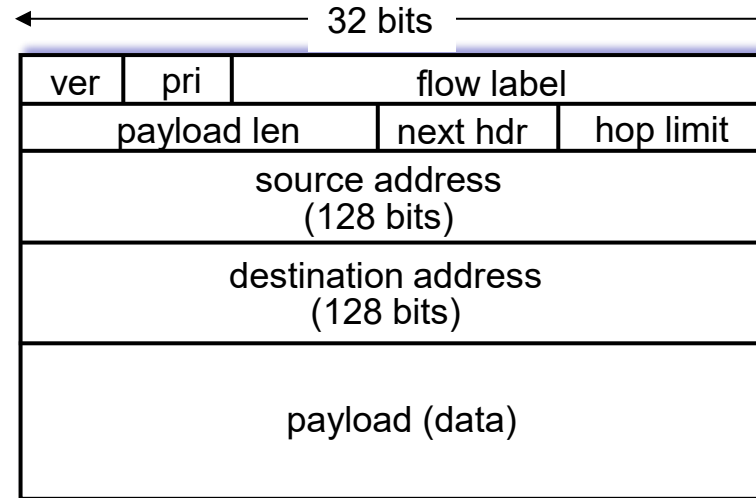
- UDP use:
 - Streaming multimedia apps (loss tolerant, rate sensitive)
 - DNS
 - DHCP
 - HTTP/3
- If reliable transfer needed over UDP (e.g., HTTP/3):
 - Add needed reliability at application layer
 - Add congestion control at application layer

UDP Segment Header

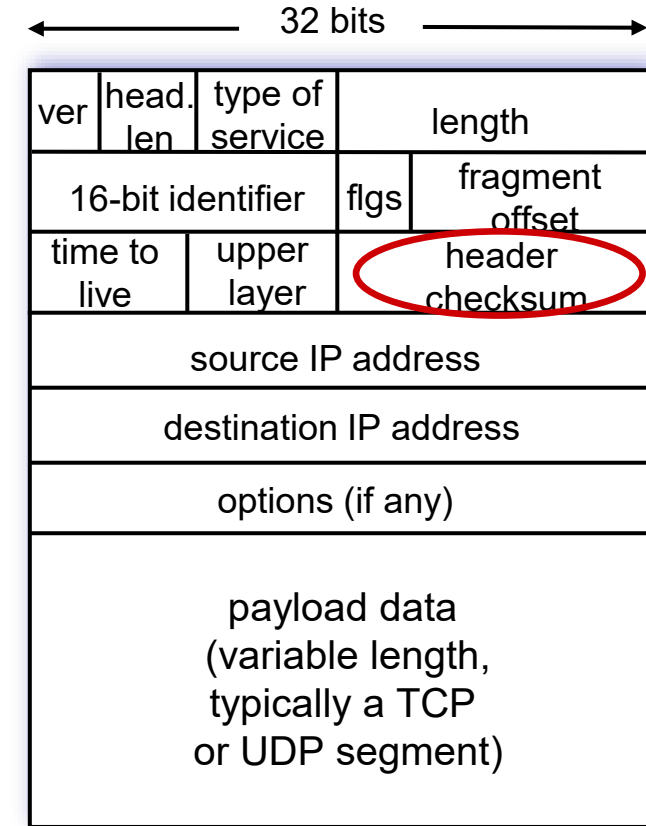
- Checksum covers UDP segment + IP pseudoheader
 - Optional in IPv4, compulsory in IPv6



UDP segment format



IPv6 datagram format



IPv4 datagram format

UDP Checksum

Suppose a segment consists of these bits: 1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 1 0 1 0 1

Diagram illustrating the wraparound checksum calculation:

Input data (16 bits): 1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0

Sum (16 bits): 1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1

Wraparound: The first bit of the sum (1) is wrapped around and added to the first bit of the data sequence (1).

Resulting checksum (16 bits): 0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1

Diagram illustrating the carry propagation in a ripple-carry adder. The diagram shows the addition of two 16-bit numbers, with the carry propagating from right to left.

The input numbers are:

- 1110011001100110
- 1101010101010101

The sum is calculated as:

1101110111011100

The carry is shown as:

0100010001000111

The final sum, including the carry, is:

sum 1111111111111111

[illegible]

UDP checksum

Goal: Detect errors (i.e., flipped bits) in transmitted segment

Sender:

- Treat contents of UDP segment (including UDP header fields and IP addresses) as sequence of 16-bit integers
- **Checksum:** addition (one's complement sum) of segment content
- Checksum value put into UDP checksum field

Receiver:

- Compute the addition of the segment contents including the checksum
 - Check if computed sum equals to all 1's:
 - Not all 1's - error detected
 - All 1's - no error detected. **But maybe errors nonetheless?**
-

UDP Checksum

Suppose a segment consists of these bits: 1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1

	1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0														
	1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1														
wraparound	1	1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1													
sum	1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0														
checksum	0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1														

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0		0	1
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1		1	0
	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1		
	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0			
	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1			
sum	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1			

Even though numbers have changed (bit flips), no change in checksum!

Summary: UDP

- “No frills” protocol:
 - Segments may be lost, delivered out of order
 - Best effort service: “send and hope for the best”
 - UDP has its plusses:
 - No setup/handshaking needed (no RTT incurred)
 - Helps with reliability (checksum)
 - Build additional functionality on top of UDP in application layer (e.g., HTTP/3)
-